

ГУАП

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

старший преподаватель
должность, уч. степень, звание

подпись, дата

Д.О. Шевяков
инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №5

СОЗДАНИЕ И НАСТРОЙКА СИСТЕМЫ УПРАВЛЕНИЯ
ОБОРУДОВАНИЕМ НА БАЗЕ ПЛАТФОРМЫ ИНТЕРНЕТА
ВЕЩЕЙ

по курсу: Интернет вещей

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № 4326

подпись, дата

Г. С. Томчук
инициалы, фамилия

Санкт-Петербург 2026

1 Цель работы

Приобретение навыков ограничения получения данных и приведения их к необходимому формату.

2 Задание

Модифицировать функции подключения вещей таким образом, чтобы все приходящие данные проверялись. Входящие данные иметь, как минимум два различных формата, включая строку. Должно быть использовано хотя бы одно регулярное выражение.

Вариант задания - Умная теплица.

3 Выполнение задания

Основной целью задания исключение ситуаций, когда неверный формат данных мог бы привести к некорректному изменению состояния устройств или сбою в работе системы.

В соответствии с заданием были обработаны как минимум два различных формата входящих данных: числовой и строковый с ограничениями. Для числовых параметров (калибровка датчиков, установка мощности устройств) использована конструкция try except с функцией float(). Если преобразование невозможно, значение игнорируется, а в консоль выводится сообщение об ошибке. Пример такой проверки приведён на рисунке 1 (фрагмент кода метода _apply_control класса Actuator).

```
def _apply_control(self, request: Request) -> None: 1 usage  ▲ Grigory Tomchuk *
    p = self.control_prefix

    power_kw = request.args.get(f"{p}_power_cmd", "").strip()
    if power_kw:
        try:
            self.set_power(float(power_kw.replace( _old: " ", _new: ".")))
        except ValueError:
            print(f"[LOG] Ошибка: мощность {self.name} – некорректное число '{power_kw}'")

    action = request.args.get(f"{p}_action", "").strip()
    if action:
        if re.match( pattern: r'^[a-zA-Za-яА-Я0-9_\\-]+$ ', action):
            self.apply_action(action)
        else:
            print(
                f"[LOG] Ошибка: действие '{action}' для {self.name} содержит недопустимые символы. Разрешены буквы, цифры, '_', '-'")

    st = request.args.get(f"{p}_power_state", "").strip().lower()
    if st:
        if re.match( pattern: r'^[on|off|1|0|true|false]$ ', st):
            if st in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(f"[LOG] Ошибка: неверное состояние питания '{st}' для {self.name}")
```

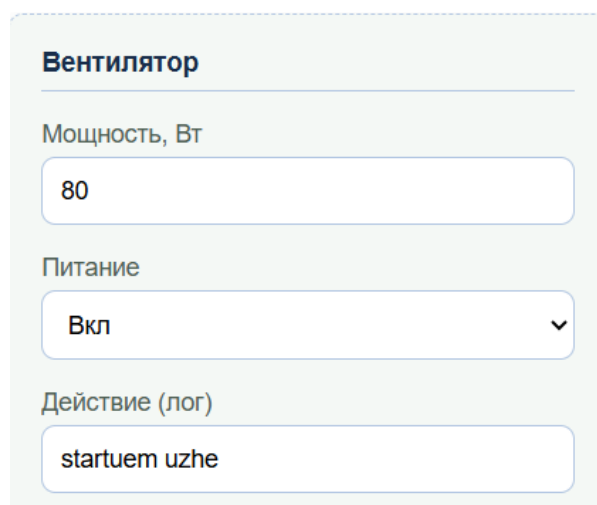
Рисунок 1 – Проверки для класса Actuator

Для строковых параметров (питание устройств, действие, режим контроллера, команда базы данных) применены регулярные выражения (модуль re). Каждый строковый параметр проверяется на соответствие допустимому набору символов или конкретным значениям. Например, для поля power_state разрешены только варианты on/off/1/0/true/false (независимо от регистра), а для поля action – буквы (в том числе русские), цифры, символы подчёркивания и дефиса. Регулярное выражение r'^[a-zA-Za-яА-Я0-9_\\-]+\$'

гарантирует, что в действии не будет пробелов или чего-то другого неприличного.

Все модификации затронули следующие классы в файле `devices.py`: `Sensor`, `Actuator`, `MainControlUnit` и `Database`. В каждом из них был доработан метод `_apply_control`, который теперь перед применением команды проверяет каждый полученный параметр.

Если данные не проходят проверку, состояние устройства не изменяется, а в лог сервера записывается предупреждение с указанием причины (неверный формат, недопустимое значение). На рисунках 2-3 показан пример работы системы: при отправке с интерфейса некорректного значения (например, “startuem uzhe” с пробелом в поле действия вентилятора) сервер отвергает команду и выводит сообщение об ошибке, при этом само устройство сохраняет прежнее состояние.



Вентилятор

Мощность, Вт

80

Питание

Вкл

Действие (лог)

startuem uzhe

Рисунок 2 – Установка значений в веб-интерфейсе

```
Мощность устройства Вентилятор (ID: 101) установлена на 80.0 Вт
Ошибка: действие 'startuem uzhe' для Вентилятор содержит недопустимые символы. Разрешены буквы, цифры, '_', '-'
Вентилятор (ID: 101) включён
```

Рисунок 3 – Лог ошибки

Примеры обработки ошибок для объектов класса `Sensor`, приведены на рисунках 4, 5 и 6, а для `Actuator` и `DataBase` на рисунках 6 и 7 соответственно.

```
[LOG] Датчик температуры (ID: 1) включён  
[LOG] Ошибка: калибровка Датчик температуры – некорректное число 'автобус'  
[LOG] Управляющая команда для датчика Датчик температуры: значение 0.0 °C, статус включения True  
[LOG] Датчик влажности воздуха (ID: 2) включён
```

Рисунок 4 – Обработка ошибок для датчика температуры

```
Датчик влажности воздуха (ID: 2) включён  
Ошибка: калибровка Датчик влажности воздуха – некорректное число 'кошка'  
Управляющая команда для датчика Датчик влажности воздуха: значение 0.0 %, статус включения True
```

Рисунок 5 – Обработка ошибок для датчика влажности воздуха

```
Датчик влажности почвы (ID: 3) включён  
Датчик Датчик влажности почвы (ID: 3) откалиброван на значение 90.0 %  
Управляющая команда для датчика Датчик влажности почвы: значение 90.0 %, статус включения True  
Насос полива (ID: 101) включён
```

Рисунок 5 – Обработка ошибок для датчика влажности воздуха

```
Ошибка: мощность Насос полива – некорректное число 'четыре'  
Ошибка: действие 'выбросить мусор' для Насос полива содержит недопустимые символы. Разрешены буквы, цифры, '_', '-'  
Насос полива (ID: 102) включён
```

Рисунок 6 – Обработка ошибок для Насоса полива

```
[LOG] БД: принята управляющая команда 'checkpoint'  
[LOG] Подключение к БД выполнено – сервер успешно запущен
```

Рисунок 7 – Обработка ошибок базы данных

Листинг программы представлен в Приложениях к лабораторной работе.

Вывод

В ходе выполнения лабораторной работы была модифицирована система управления оборудованием умной теплицы: во все функции подключения объектов добавлена проверка входящих данных на стороне сервера. Реализована валидация двух форматов данных – числового (с помощью конструкции try-except и приведения к float) и строкового (с применением регулярных выражений). Использование регулярных выражений позволило ограничивать допустимые значения для таких параметров, как питание устройств, действия, режим контроллера и команды базы данных.

В результате система стала устойчивой к ошибочным или некорректным запросам: неверные данные игнорируются, состояние устройств не изменяется, а в лог сервера выводятся предупреждения.

ПРИЛОЖЕНИЯ
ПРИЛОЖЕНИЕ А
Листинг app.py

```
import logging

from flask import Flask, jsonify, render_template, request

from devices import Actuator, Database, MainControlUnit, Sensor

app = Flask(__name__)

temperature_sensor = Sensor(1, "Датчик температуры", "temperature", "°C",
"temperature")
humidity_sensor = Sensor(2, "Датчик влажности воздуха", "humidity", "%",
"humidity")
soil_sensor = Sensor(3, "Датчик влажности почвы", "soil_moisture", "%", "soil")
fan = Actuator(101, "Вентилятор", "fan", 120.0)
pump = Actuator(102, "Насос полива", "pump", 80.0)
controller = MainControlUnit()
database = Database("sqlite:///greenhouse.db", "./data")

temperature_sensor.turn_on()
humidity_sensor.turn_on()
soil_sensor.turn_on()
fan.turn_on()
pump.turn_on()

@app.route("/")
def index():
    return render_template("dashboard.html")

@app.route("/connect/temperature")
def connect_temperature():
    return jsonify(temperature_sensor.connect())

@app.route("/connect/humidity")
def connect_humidity():
    return jsonify(humidity_sensor.connect())

@app.route("/connect/soil")
def connect_soil():
    return jsonify(soil_sensor.connect())
```

```

@app.route("/connect/fan")
def connect_fan():
    return jsonify(fan.connect())

@app.route("/connect/pump")
def connect_pump():
    return jsonify(pump.connect())

@app.route("/connect/controller")
def connect_controller():
    return jsonify(controller.connect())

@app.route("/connect/database")
def connect_database():
    return jsonify(database.connect())

@app.route("/connect/command", methods=["GET"])
def connect_command():
    """Применить управляющие параметры ко всем вещам."""
    return jsonify(
        {
            "temperature": temperature_sensor.connect(request),
            "humidity": humidity_sensor.connect(request),
            "soil": soil_sensor.connect(request),
            "fan": fan.connect(request),
            "pump": pump.connect(request),
            "controller": controller.connect(request),
            "database": database.connect(request),
        }
    )

if __name__ == "__main__":
    logging.getLogger("werkzeug").setLevel(logging.ERROR)
    app.run(debug=True)

```


ПРИЛОЖЕНИЕ В

Листинг `devices.py`

```
import re
from abc import ABC, abstractmethod
from datetime import datetime
from typing import Dict, List, Optional, Union

from werkzeug.wrappers import Request

class Device(ABC):
    def __init__(self, device_id: int, name: str, status: bool = False):
        self.id = device_id
        self.name = name
        self.status = status

    def turn_on(self):
        self.status = True
        print(f'[LOG] {self.name} (ID: {self.id}) включён')

    def turn_off(self):
        self.status = False
        print(f'[LOG] {self.name} (ID: {self.id}) выключен')

    def get_status(self) -> bool:
        print(f'[LOG] Запрос статуса {self.name} (ID: {self.id}) -> {self.status}')
        return self.status

    @abstractmethod
    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[int, float, str, bool]]:
        """Мониторинг (request is None) или применение команд (передан
        request)."""

class Sensor(Device):
    def __init__(
        self,
        device_id: int,
        name: str,
        sensor_type: str,
        unit: str,
        control_prefix: str,
    ):
        pass
```

```

    super().__init__(device_id, name)
    self.type = sensor_type
    self.unit = unit
    self.control_prefix = control_prefix
    self.value: float = 0.0

def measure(self):
    self.value = 22.5
    print(
        f"[LOG] Датчик {self.name} (ID: {self.id}) измерил: {self.value}
{self.unit}"
    )

def calibrate(self, value: float):
    self.value = value
    print(
        f"[LOG] Датчик {self.name} (ID: {self.id}) откалиброван на значение
{value} {self.unit}"
    )

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix
    power_raw = request.args.get(f"{p}_power", "").strip().lower()
    if power_raw:
        # Допустимо on, off, 1, 0, true, false
        if re.match(r'^(on|off|1|0|true|false)$', power_raw):
            if power_raw in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(
                f"[LOG] Ошибка: недопустимое значение power '{power_raw}' для
{self.name}. Допустимо: on/off/1/0/true/false"
            )

    raw_cal = request.args.get(f"{p}_calibrate", "").strip()
    if raw_cal:
        try:
            # Заменяем запятую на точку и преобразуем в float
            value = float(raw_cal.replace(",", "."))
            self.calibrate(value)
        except ValueError:
            print(f"[LOG] Ошибка: калибровка {self.name} – некорректное число
'{raw_cal}'")

```

```

def connect(
    self, request: Optional[Request] = None
) -> Dict[str, Union[int, float, str, bool]]:
    if request is not None:
        self._apply_control(request)
        print(
            f"[LOG] Управляющая команда для датчика {self.name}: "
            f"значение {self.value} {self.unit}, статус включения {self.status}"
        )
    else:
        print(
            f"[LOG] Опрос датчика {self.name}: текущее значение {self.value} "
            f"{self.unit}"
        )
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "unit": self.unit,
        "value": self.value,
        "status": self.status,
    }

class Actuator(Device):
    def __init__(
        self, device_id: int, name: str, actuator_type: str, power: float = 0.0
    ):
        super().__init__(device_id, name)
        self.type = actuator_type
        self.power = power
        self.control_prefix = actuator_type

    def apply_action(self, action: str):
        print(
            f"[LOG] Исполнительное устройство {self.name} (ID: {self.id}) "
            f"выполняет действие: {action}"
        )

    def set_power(self, level: float):
        self.power = level
        print(
            f"[LOG] Мощность устройства {self.name} (ID: {self.id}) установлена "
            f"на {level} Вт"
        )

```

```

def _apply_control(self, request: Request) -> None:
    p = self.control_prefix

    power_kw = request.args.get(f'{p}_power_cmd', "").strip()
    if power_kw:
        try:
            self.set_power(float(power_kw.replace(",", ".")))
        except ValueError:
            print(f'[LOG] Ошибка: мощность {self.name} – некорректное число '
                  f'{power_kw}')

    action = request.args.get(f'{p}_action', "").strip()
    if action:
        if re.match(r'^[a-zA-Za-яA-Я0-9_-]+$', action):
            self.apply_action(action)
        else:
            print(
                f'[LOG] Ошибка: действие '{action}' для {self.name} содержит '
                f'недопустимые символы. Разрешены буквы, цифры, '_', '-'')

    st = request.args.get(f'{p}_power_state', "").strip().lower()
    if st:
        if re.match(r'^(on|off|1|0|true|false)$', st):
            if st in ("on", "1", "true"):
                self.turn_on()
            else:
                self.turn_off()
        else:
            print(f'[LOG] Ошибка: неверное состояние питания '{st}' для '
                  f'{self.name}')

def connect(
    self, request: Optional[Request] = None
) -> Dict[str, Union[int, float, str, bool]]:
    if request is not None:
        self._apply_control(request)
        print(
            f'[LOG] Управление {self.name}: мощность {self.power} Вт, статус '
            f'{self.status}'
        )
    else:
        print(
            f'[LOG] Опрос исполнительного устройства {self.name}: '
            f'{self.power} Вт, статус: {self.status}'

```

```

    )
    return {
        "id": self.id,
        "name": self.name,
        "type": self.type,
        "power": self.power,
        "status": self.status,
    }

```

```

class MainControlUnit:

```

```

    def __init__(self):
        self.mode = "auto"
        self.schedule: Dict[str, str] = {}
        self.alerts: List[str] = []

```

```

    def process_data(self, sensor_data: Dict[int, float]):
        print(f"[LOG] Контроллер обрабатывает данные с датчиков:
{sensor_data}")

```

```

    def send_command(self, actuator: Actuator, command: str):
        print(
            f"[LOG] Контроллер отправляет команду '{command}' устройству
{actuator.name} (ID: {actuator.id})"
        )
        actuator.apply_action(command)

```

```

    def check_alerts(self):
        print("[LOG] Контроллер проверяет наличие тревог...")
        if self.alerts:
            print(f"[LOG] Активные тревоги: {self.alerts}")
        else:
            print("[LOG] Тревог нет")

```

```

    def _apply_control(self, request: Request) -> None:
        mode = request.args.get("controller_mode", "").strip().lower()
        if mode:
            if re.match(r'^(auto|manual)$', mode):
                self.mode = mode
                print(f"[LOG] Контроллер: режим установлен в {self.mode}")
            else:
                print(f"[LOG] Ошибка: неверный режим '{mode}'. Допустимо: auto /
manual")

```

```

        clear = request.args.get("controller_clear_alerts", "").strip().lower()
        if clear:

```

```

        if re.match(r'^(1|true|yes|on)$', clear):
            self.alerts = []
            print("[LOG] Контроллер: список тревог очищен")
        else:
            print(f"[LOG] Ошибка: неверное значение clear_alerts '{clear}'.  
Игнорируем.")

    def connect(
        self, request: Optional[Request] = None
    ) -> Dict[str, Union[str, int, List[str]]]:
        if request is not None:
            self._apply_control(request)
        print(
            f"[LOG] Подключение к контроллеру выполнено, режим: {self.mode},  
количество тревог: {len(self.alerts)}"
        )
        return {
            "mode": self.mode,
            "alerts_count": len(self.alerts),
            "alerts": list(self.alerts),
        }

class Database:
    def __init__(self, connection_string: str, storage_path: str):
        self.connection_string = connection_string
        self.storage_path = storage_path
        self.records_today: int = 0

    def save_reading(self, sensor_id: int, value: float, timestamp: datetime):
        self.records_today += 1
        print(
            f"[LOG] БД: сохранено показание датчика {sensor_id} = {value} в  
{timestamp}"
        )

    def save_event(self, device_id: int, action: str, timestamp: datetime):
        self.records_today += 1
        print(
            f"[LOG] БД: сохранено событие устройства {device_id}: {action} в  
{timestamp}"
        )

    def get_history(
        self, start: datetime, end: datetime, device_id: Optional[int] = None
    ):

```

```

    print(
        f"[LOG] БД: запрос истории с {start} по {end} для устройства
        {device_id if device_id else 'всех'}"
    )

    return []

def _apply_control(self, request: Request) -> None:
    cmd = request.args.get("database_command", "").strip()
    if cmd:
        if re.match(r'^[a-zA-Z0-9_\-]+$', cmd):
            print(f"[LOG] БД: принята управляющая команда '{cmd}'")
        else:
            print(
                f"[LOG] Ошибка: команда БД '{cmd}' содержит недопустимые
                символы. Разрешены буквы, цифры, '_', '-'")

    def connect(self, request: Optional[Request] = None) -> Dict[str, Union[str,
int]]:
    if request is not None:
        self._apply_control(request)
    data = {
        "connection": "ok",
        "storage_path": self.storage_path,
        "records_today": self.records_today,
    }
    print(
        f"[LOG] Подключение к БД выполнено, записей за сегодня:
        {data['records_today']}, путь: {data['storage_path']}"
    )
    return data

```